

ML Classification Algorithms

Study sheet — cliff notes, use cases, scikit-learn code, visuals

Naive Bayes

Probability-based classifier

Probabilistic

CLIFF NOTE

Uses Bayes' theorem to predict class probabilities. Assumes all features are independent — which is rarely true in real life, hence "naive". Despite this, it works surprisingly well for text data.

USE CASE

Email spam detection

Given the words in an email, what's the probability it's spam? Each word is treated independently, e.g. $P(\text{spam} \mid \text{'free', 'win', 'click'})$.

CODE

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVectorizer

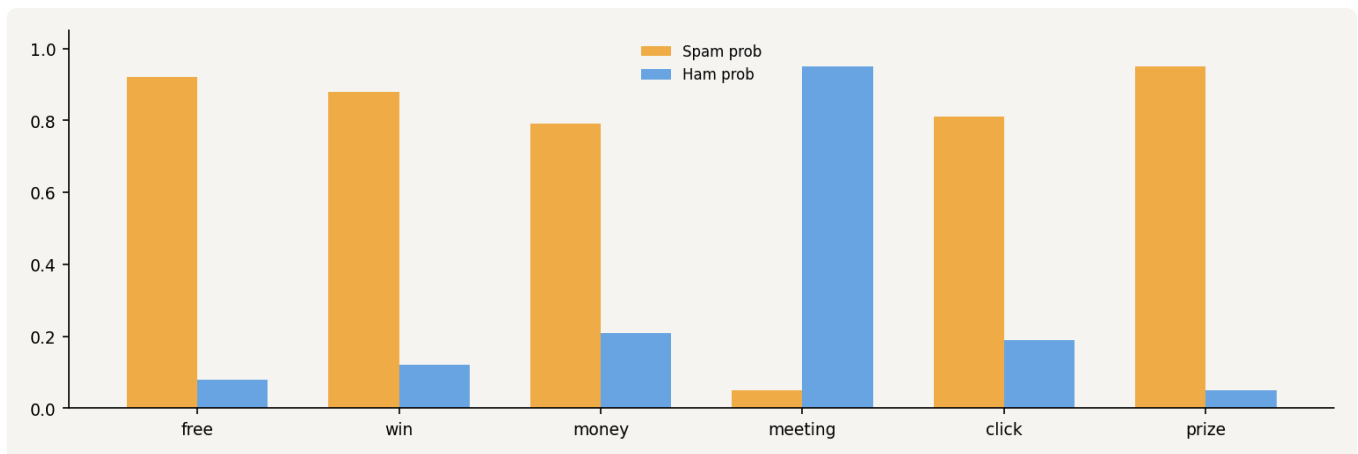
emails = ["win free money now", "meeting at 3pm",
          "click here for prize"]
labels = [1, 0, 1] # 1=spam, 0=ham

vec = CountVectorizer()
X = vec.fit_transform(emails)

model = MultinomialNB()
model.fit(X, labels)

pred = model.predict(vec.transform(["free prize click"]))
print(pred) # [1] → spam
```

VISUAL — WORD PROBABILITIES — SPAM VS HAM



Logistic Regression

Linear

Linear decision boundary

CLIFF NOTE

Fits a linear boundary between classes, then squashes the output through a sigmoid function to get a probability between 0 and 1. Fast, interpretable, and a great baseline for any classification problem.

USE CASE

Customer churn prediction

Given features like usage frequency, support tickets, and contract length — what's the probability a customer will cancel?

CODE

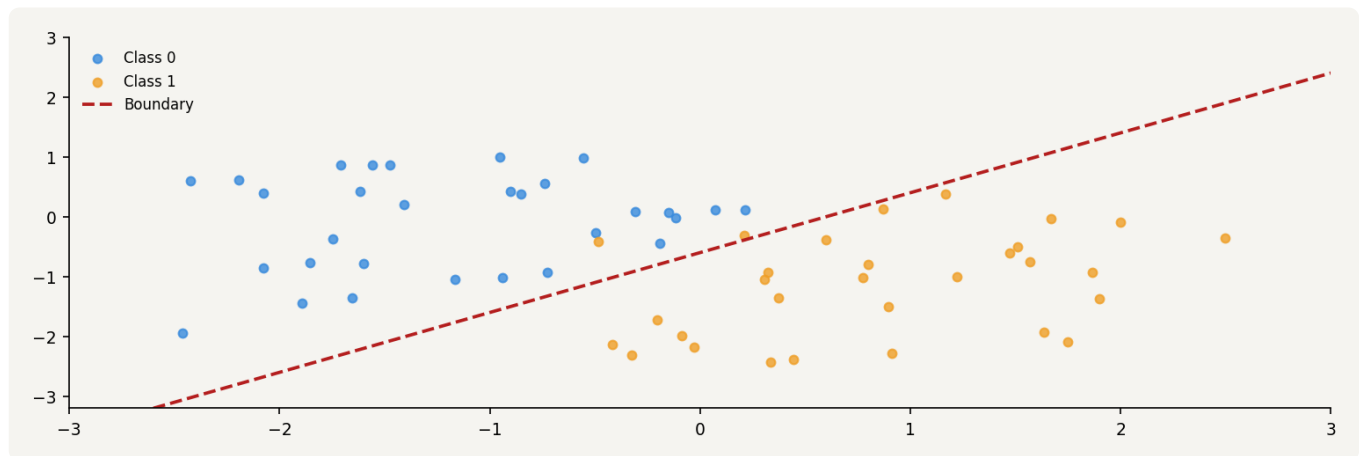
```
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=200, n_features=2,
                          n_redundant=0, random_state=42)

model = LogisticRegression()
model.fit(X, y)

print(model.predict([[0.5, -1.2]]))
# → [0] or [1]
print(model.predict_proba([[0.5, -1.2]]))
# → [[0.73, 0.27]]
```

VISUAL — 2D SCATTER WITH DECISION BOUNDARY



Decision Trees

Tree-based

Rule-based splitting

CLIFF NOTE

Recursively splits data into branches by asking yes/no questions on features. Very interpretable — you can literally read the rules. Prone to overfitting without pruning.

USE CASE

Loan approval

If income > 50k AND credit score > 700 AND debt ratio < 0.3
→ approve. Each split mirrors how a loan officer actually thinks.

CODE

```
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.datasets import load_iris

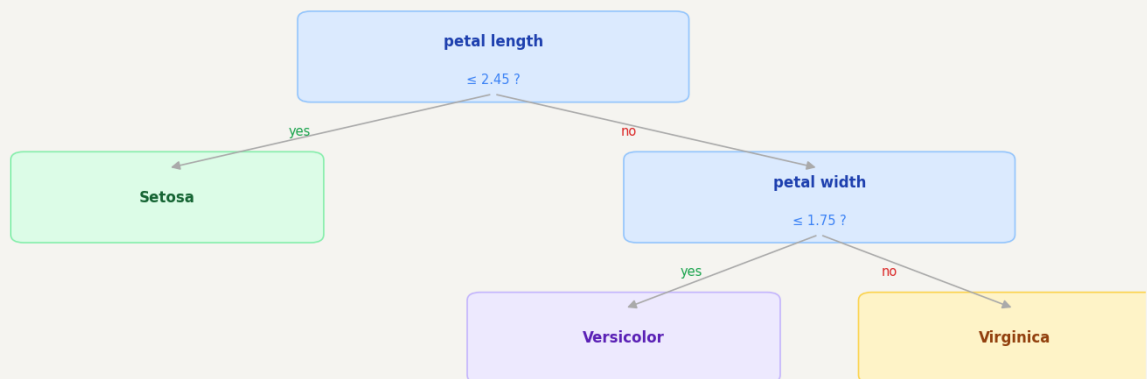
X, y = load_iris(return_X_y=True)

model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X, y)

rules = export_text(model,
    feature_names=load_iris().feature_names)
print(rules)

print(model.predict([[5.1, 3.5, 1.4, 0.2]]))
# → [0] = setosa
```

VISUAL — IRIS DECISION TREE STRUCTURE



K-Nearest Neighbors

Instance-based

Majority vote from neighbors

CLIFF NOTE

No training phase — at prediction time it finds the K closest data points and takes a majority vote. Simple and intuitive, but slow on large datasets and sensitive to irrelevant features.

USE CASE

Movie recommendations

Find the K users most similar to you based on viewing history.
Recommend what they watched that you haven't.

CODE

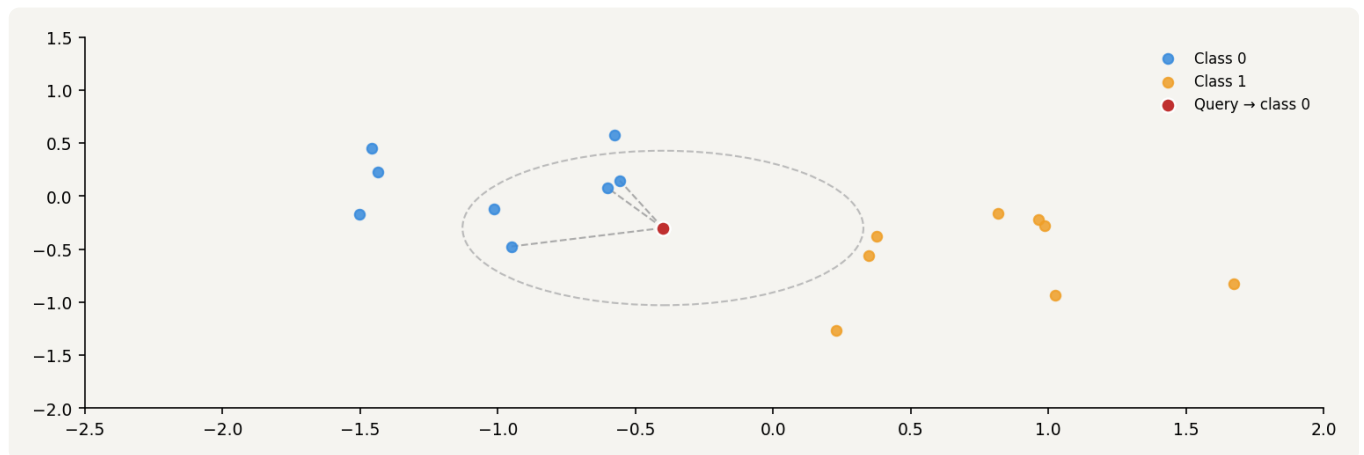
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=300, n_features=2,
                          n_redundant=0, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

print(model.score(X_test, y_test)) # accuracy
print(model.predict([[0.0, 0.5]])) # class
```

VISUAL — K=3 NEAREST NEIGHBORS — QUERY → CLASS 0



Support Vector Machines

Margin-based

Maximum margin classifier

CLIFF NOTE

Finds the hyperplane that maximises the margin between classes. Uses only the nearest data points (support vectors) to define the boundary. Kernels let it handle non-linear data.

USE CASE

Handwritten digit classification

Works well in high-dimensional spaces with a clear margin. A classic approach for MNIST digit recognition.

CODE

```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_classification

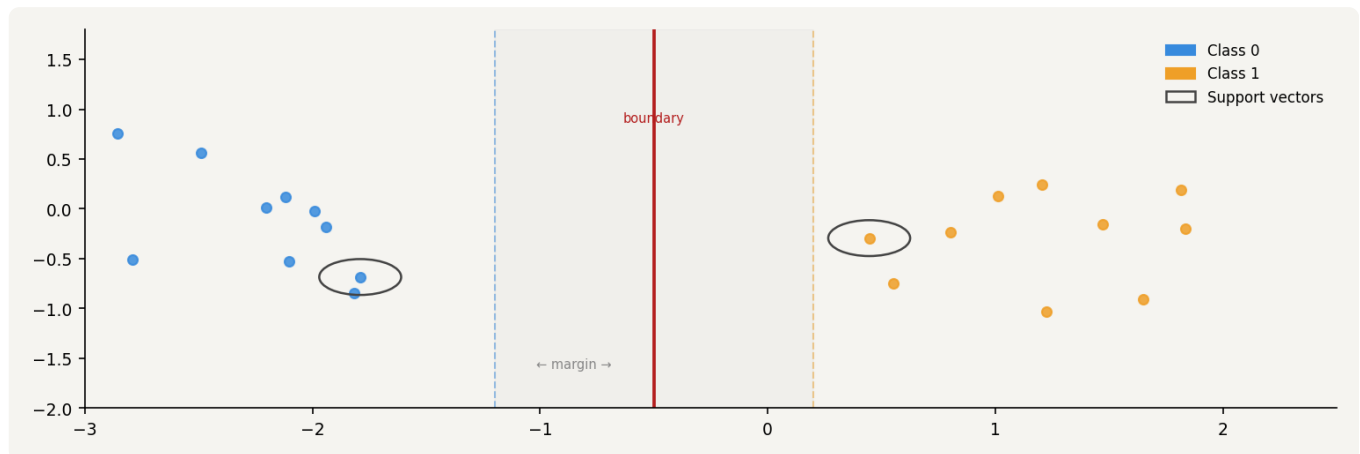
X, y = make_classification(n_samples=300, n_features=2,
                          n_redundant=0, random_state=42)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = SVC(kernel='rbf', C=1.0, probability=True)
model.fit(X_scaled, y)

print(model.predict(scaler.transform([[0.5, -1.0]])))
print(f"Support vectors: {model.support_vectors_.shape[0]}")
```

VISUAL — MAXIMUM MARGIN + SUPPORT VECTORS



Neural Networks

Deep learning

Layered feature learning

CLIFF NOTE

Stacked layers of neurons learn increasingly abstract features. Each layer transforms input via weights + activation functions. The network adjusts weights through backpropagation during training.

USE CASE

Sentiment analysis

Classify customer reviews as positive or negative. The layers learn to detect words, phrases, and context automatically.

CODE

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_classification

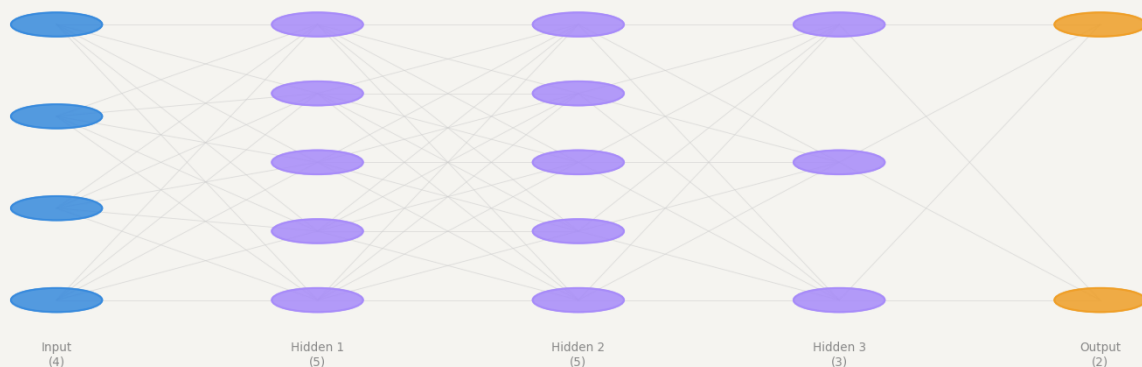
X, y = make_classification(n_samples=500, n_features=4,
                          n_redundant=0, random_state=42)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 2 hidden layers: 64 and 32 neurons
model = MLPClassifier(hidden_layer_sizes=(64, 32),
                      activation='relu', max_iter=500,
                      random_state=42)
model.fit(X_scaled, y)

print(model.score(X_scaled, y)) # training accuracy
```

VISUAL — NEURAL NETWORK LAYER ARCHITECTURE



Random Forest

Ensemble

Ensemble of diverse trees

CLIFF NOTE

Builds hundreds of decision trees on random subsets of data and features, then aggregates their votes. The diversity between trees reduces overfitting. One of the most reliable all-round classifiers.

USE CASE

Medical diagnosis

Predict whether a patient has a disease based on many lab values. The ensemble smooths out noise in individual test results.

CODE

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

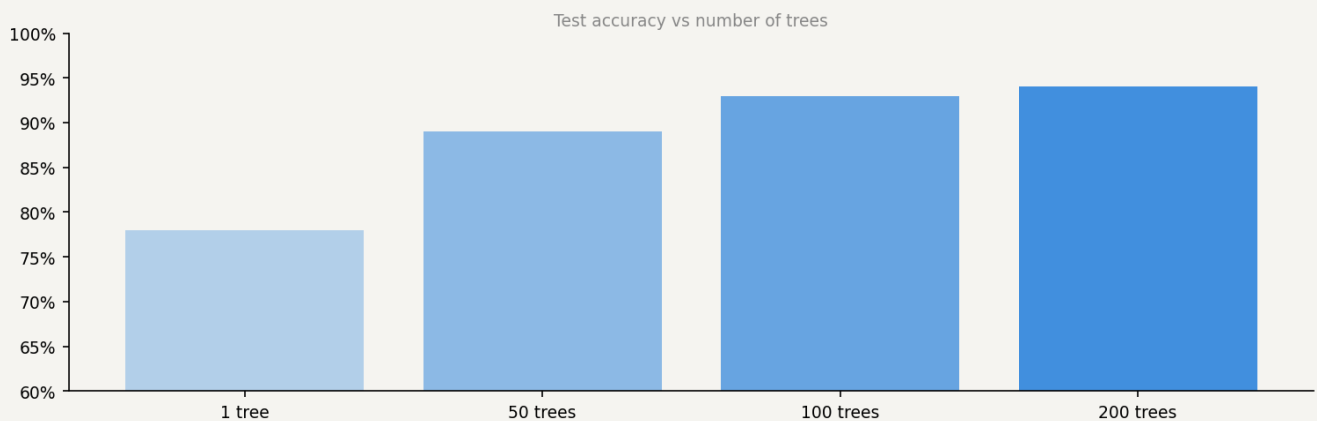
X, y = make_classification(n_samples=500, n_features=10,
                           n_redundant=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

print(f"Accuracy: {model.score(X_test, y_test):.3f}")

importances = model.feature_importances_
top3 = sorted(enumerate(importances), key=lambda x:-x[1])[:3]
print(top3) # top 3 most important features
```

VISUAL — ACCURACY VS NUMBER OF TREES



XGBoost / Gradient Boosting

Ensemble

Sequential error correction

CLIFF NOTE

Builds trees one at a time, where each tree focuses on correcting the errors of the previous. Uses gradient descent to minimise a loss function. Fast, powerful, regularised — Kaggle's favourite.

USE CASE

Credit scoring

Predict whether a loan applicant will default. Handles missing data natively, works well with mixed feature types.

CODE

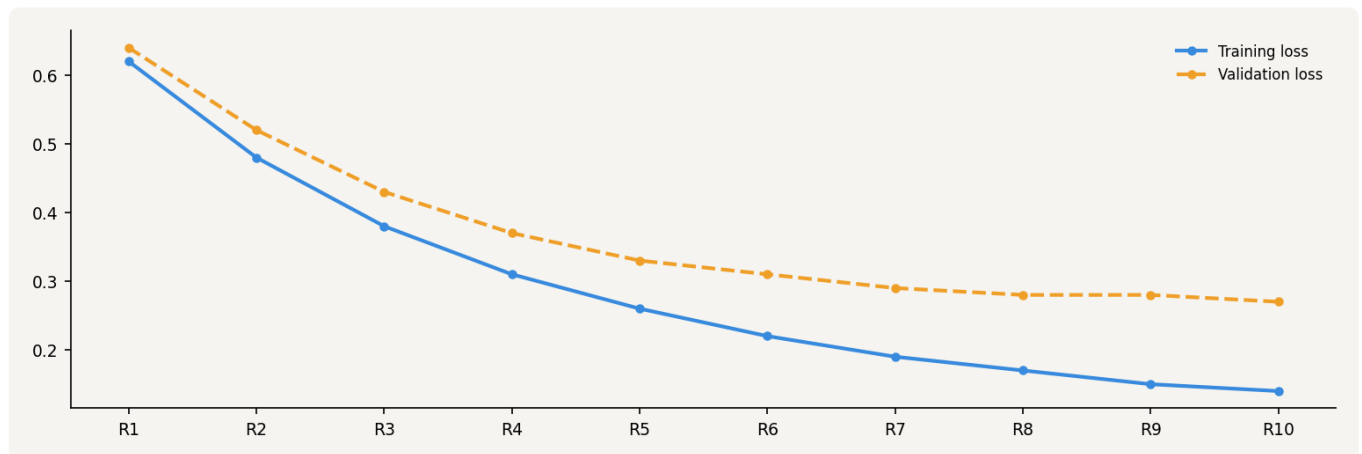
```
# pip install xgboost
from xgboost import XGBClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=500, n_features=10,
                          n_redundant=2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2)

model = XGBClassifier(n_estimators=100, learning_rate=0.1,
                      max_depth=4, eval_metric='logloss',
                      random_state=42)
model.fit(X_train, y_train,
          eval_set=[(X_test, y_test)], verbose=False)

print(f"Accuracy: {model.score(X_test, y_test):.3f}")
```

VISUAL — TRAINING VS VALIDATION LOSS PER ROUND



LDA / QDA

Statistical

Discriminant analysis

CLIFF NOTE

LDA finds a linear combination of features that best separates classes (assumes equal covariance). QDA is the flexible variant allowing each class its own covariance. Both are fast and work well with small datasets.

USE CASE

Face recognition

LDA (Fisherfaces) was a classic approach — project face images onto the dimensions that best separate identities.

CODE

```
from sklearn.discriminant_analysis import (
    LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis
)
from sklearn.datasets import load_wine

X, y = load_wine(return_X_y=True)

lda = LinearDiscriminantAnalysis()
lda.fit(X, y)
print(f"LDA accuracy: {lda.score(X, y):.3f}")

qda = QuadraticDiscriminantAnalysis()
qda.fit(X, y)
print(f"QDA accuracy: {qda.score(X, y):.3f}")

X_2d = lda.transform(X)  # collapse to (n_classes-1) dims
print(X_2d.shape)        # (178, 2)
```

VISUAL — 3-CLASS SEPARATION + LD1 AXIS

